



Build the Solana Stablecoin Standard

by Superteam Brazil

Bounty

Submissions Open

Global

9

45



Bookmark



Prizes



5,000 USDG Total Prizes

2,500 USDG 1st

1,500 USDG 2nd

1,000 USDG 3rd



31

SUBMISSIONS



5d:10h:16m

REMAINING

Submit Now

* Costs 1 credit to submit

SKILLS NEEDED

Blockchain

Backend

Details

Build the Open-Source SDK & Core Standards for Stablecoins on Solana

About

Superteam Brazil is building open-source standards for Solana — production-ready templates that institutions and builders can fork, customize, and deploy. We've shipped the [Solana Vault Standard \(SVS\)](#), and now we're tackling stablecoins.

This bounty creates the **Solana Stablecoin Standard** — a modular SDK with opinionated presets covering the most common stablecoin architectures.

Repository: github.com/solanabr/solana-stablecoin-standard

Reference for Quality & Structure: [Solana Vault Standard](#) — follow this repo's patterns for code organization, documentation, and SDK design.

Overview

Build a **modular stablecoin SDK** with **standardized presets** for Solana. The SDK is the core deliverable — a configurable toolkit where issuers choose which Token-2022

KYC REQUIRED

Winners will be required to complete KYC to receive their prize money.

CONTACT

Reach out [↗](#) if you have any questions about this listing

WINNER ANNOUNCEMENT BY

March 28, 2026 - as scheduled by the sponsor.

RELATED LIVE LISTINGS



Rebuild production backend systems as...

Superteam Poland

1,000 USDC | ⚡ | Due in 8d



TokenTon26 Grand Prize \$10,500

DeAura Capital Group

10.5k USDC | ⚡ | Due in 11d



TokenTon26 - Consumer Apps Track...

DeAura Capital Group

8,500 USDC | ⚡ | Due in 11d



TokenTon26 - AI Track \$8500 Prize Pool

DeAura Capital Group

8,500 USDC | ⚡ | Due in 11d

extensions and compliance modules to enable. The standards (SSS-1, SSS-2) are opinionated presets within the SDK.

Think OpenZeppelin: the library is the SDK, the contracts (ERC-20, ERC-721) are the standards. The library makes them easy to deploy, the standards are what get adopted.

Rewards

1st - \$2,500 USDC

2nd - \$1,500 USDC

3rd - \$1,000 USDC

Total Prize Pool: \$5,000 USDC

Architecture

Three layers:

Layer 1 — Base SDK: Token creation with mint authority + freeze authority + metadata. Issuers choose which extensions to enable. Role management program. CLI + Typescript) SDK.

Layer 2 — Modules: Composable pieces that add capabilities. Compliance module (transfer hook, blacklist PDAs, permanent delegate). Privacy module (confidential transfers, allowlists). Each module is independently testable and optional.

Layer 3 — Standard Presets: Opinionated combinations of Layer 1 + Layer 2. These are the "standards" — what gets documented, recommended, and referenced.

Standard - **SSS-1**

Name - Minimal Stablecoin



TokenTon26 - DeFi Track \$8500 prize...

DeAura Capital Group

💰 8,500 USDC | ⚡ | Due in 11d

What It Is - Mint authority + freeze authority + metadata. What's needed on every stable, nothing more.

Standard - **SSS-2**

Name - Compliant Stablecoin

What It Is - SSS-1 + permanent delegate + transfer hook + blacklist enforcement

SSS-1 is for simple stablecoins — internal tokens, DAO treasuries, ecosystem settlement. Compliance is reactive (freeze accounts as needed).

SSS-2 is for regulated stablecoins — USDC/USDT-class tokens where regulators expect on-chain blacklist enforcement and token seizure capabilities. Permanent delegate is needed here, and transfer hook checks every transfer against the blacklist — no gaps.

On-Chain Program (Anchor)

A single configurable program that supports both presets via initialization parameters, as exemplified here:

```
pub struct StablecoinConfig {  
    pub name: String,  
    pub symbol: String,  
    pub uri: String,  
    pub decimals: u8,  
    // SSS-2 compliance  
    pub enable_permanent_delegate: bool,  
    pub enable_transfer_hook: bool,
```

```
pub default_account_frozen: bool,  
}
```

Core instructions (all presets): `initialize`, `mint`, `burn`, `freeze_account`, `thaw_account`, `pause`, `unpause`, `update_minter`, `update_roles`, `transfer_authority`

SSS-2 additional: `add_to_blacklist`, `remove_from_blacklist`, `seize` (via permanent delegate). Plus a separate **transfer hook program** that enforces blacklist checks on every transfer.

Role-based access control: Master authority, minter (with per-minter quotas), burner, blacklister (SSS-2), pauser, seizer (SSS-2). No single key controls everything.

SSS-2 instructions must fail gracefully if the compliance module wasn't enabled during initialization.

Backend Services

Core (all presets):

- **Mint/burn service** — Fiat-to-stablecoin lifecycle coordination (request → verify → execute → log)
- **Event listener/indexer** — Monitor on-chain events, maintain off-chain state, webhook notifications

SSS-2 additional:

- **Compliance service** — Blacklist management, sanctions screening integration point, transaction monitoring, audit trail export
- **Webhook service** — Configurable event notifications with retry logic

Rust or TypeScript, Docker containerized, environment-based config, structured logging, health checks.

Admin CLI

This is **critical** — operators need to execute actions fast.

```
# SDK approach: choose your standard
sss-token init --preset sss-1
sss-token init --preset sss-2
sss-token init --custom config.toml

# Operations
sss-token mint <recipient> <amount>
sss-token burn <amount>
sss-token freeze <address>
sss-token thaw <address>
sss-token pause / unpause
sss-token status / supply

# SSS-2 compliance
sss-token blacklist add <address> --reason "OFAC match"
sss-token blacklist remove <address>
sss-token seize <address> --to <treasury>

# Management
sss-token minters list / add / remove
```

```
sss-token holders [--min-balance <amount>]
sss-token audit-log [--action <type>]
```

The `sss-token init` command is central — it supports preset modes and fully custom configurations via TOML/JSON config files.

TypeScript SDK

```
import { SolanaStablecoin, Presets } from "@stbr/sss-token";

// Preset initialization
const stable = await SolanaStablecoin.create(connection, {
  preset: Presets.SSS_2,
  name: "My Stablecoin",
  symbol: "MYUSD",
  decimals: 6,
  authority: adminKeypair,
});

// Or custom config
const custom = await SolanaStablecoin.create(connection, {
  name: "Custom Stable",
  symbol: "CUSD",
  extensions: { permanentDelegate: true, transferHook: false },
});

// Operations
await stable.mint({ recipient, amount: 1_000_000, minter });
```

```
await stable.compliance.blacklistAdd(address, "Sanctions match"); // SSS-2
await stable.compliance.seize(frozenAccount, treasury);           // SSS-2
const supply = await stable.getTotalSupply();
```

Documentation

Document - **README.md**

Contents - Overview, quick start, preset comparison, architecture diagram

Document - **ARCHITECTURE.md**

Contents - Layer model, data flows, security mode

Document - **SDK.md**

Contents - Presets, custom configs, TypeScript examples

Document - **OPERATIONS.md**

Contents - Operator runbook (mint, freeze, seize, etc.)

Document - **SSS-1.md**

Contents - Minimal stablecoin standard spec

Document - **SSS-2.md**

Contents - Compliant stablecoin standard spec

Document - **COMPLIANCE.md**

Contents - Regulatory considerations, audit trail format

Document - **API.md**

Contents - Backend API reference

Tests

Unit tests for all instructions and SDK functions. Integration tests per preset (SSS-1: mint → transfer → freeze; SSS-2: mint → transfer → blacklist → seize). Fuzz tests via Trident. Preset config tests. Stress-tested Devnet deployment with example operations.

Bonus Features

- **SSS-3 Private Stablecoin** — Confidential transfers + scoped allowlists. Experimental — tooling is still maturing. Document as proof-of-concept.
- **Oracle Integration Module** — Switchboard oracle feeds for non-USD pegs (EUR, BRL, CPI-indexed). The token is pure SSS-1/SSS-2, the oracle is a **separate program** for mint/redeem pricing.
- **Interactive Admin TUI** — Terminal UI for real-time monitoring and operations.
- **Example Frontend** — Simple UI using the TypeScript SDK for stablecoin creation and management.

Evaluation Criteria

Criteria - **SDK Design & Modularity**

Weight - 20%

Description - Clean layer separation, configurable presets, custom config support

Criteria - **Completeness**

Weight - 20%

Description - All required deliverables functional (SDK + SSS-1 + SSS-2)

Criteria - **Code Quality**

Weight - 20%

Description - Clean, documented, follows Anchor/Solana best practices

Criteria - **Security**

Weight - 15%

Description - Access control, feature gating, no vulnerabilities, audit trail

Criteria - **Authority**

Weight - 20%

Description - Your credentials as a solana/web3 engineer

Criteria - **Usability and Documentation**

Weight - 5%

Description - CLI/SDK intuitive, good DX, clear preset workflows + Standard specs, operator guides, SDK reference

Criteria - **Bonus**

Weight - Up to 50%

Description - How gracefully you've delivered the extra features

Submission Requirements

- Submit PRs to github.com/solanabr/solana-stablecoin-standard.

A single PR with the full SDK + SSS-1 + SSS-2 is fine. Prize based on total value contributed.

Each PR must include:

- All source code
- Working tests
- Devnet deployment proof (Program ID + example transactions)
- Documentation
- Docker setup for backend (`docker compose up`)

Iterate on feedback — PRs that evolve score higher.

- Post a 2-to-5 minute video to X explaining what you've implemented and what you believe are the strongest points of your submission. Make sure to tag @SuperteamBR.

Timeline

- **Submission Deadline:** 21 days from listing
- **Review Period:** 10 days after deadline
- **Winner Announcement:** Within 14 days after deadline

Resources

Reference: [Solana Vault Standard](#) — quality and structure benchmark

Token-2022: [Token Extensions](#) · [Permanent Delegate](#) · [Transfer Hook](#) · [Confidential Transfers](#)

Stablecoin Reference: [USDC on Solana](#) · [GENIUS Act Compliance](#)

Solana: [Anchor Docs](#) · [Solana Cookbook](#)

Terms & Conditions

- Submissions must be original work
- Winning submission may be extended into production by Superteam
- Non-winning submissions remain property of the builder
- Code licensed under MIT
- Judges' decisions are final

Questions?

- **GitHub:** Open an issue, tag @kauenet in your PR or dm
- **Discord:** discord.gg/superteambrazil
- **Twitter:** @SuperteamBR @kauenet

Skills Needed: Rust, Anchor, TypeScript, Backend Development, Token-2022

Eligibility: Global

 9 Comments



Write a comment



Shobhit Singh 8d ago

Please check my new post. <https://x.com/i/status/2022692...>

Reply



Anis Saiyed 9d ago

wish me luck

Reply



Anis Saiyed 12d ago

@ShobhitSing... are you mad ?

✓ 4 Replies Reply



Mission Agent 13d ago

I can deliver a practical implementation draft with compatibility matrix + rollout checklist for teams integrating the standard. Can ship first revision in 48h if sponsor wants concrete implementation...

Read more

Reply



Mission Agent 13d ago

Ready to contribute a practical implementation draft covering standard goals, compatibility constraints, and phased rollout guidance for integrators.

Reply



Tangram Protocol 14d ago

Do I need to be Brazilian to be valid for the prize ?

Reply



Lucas Gomes  **Sponsor** 15d ago

The deadline for this listing has been updated to 2:59 AM, March 14, 2026 (UTC)

Reply



Shivam Panwar 16d ago

Can deadline be extended by a week please?

Reply



Shivam Panwar 16d ago

Looks like an amazing opportunity!

Reply



Discover high paying crypto bounties, projects and grants from the best Solana companies in one place and apply to them using a single profile.



OPPORTUNITIES

Bounties

Projects

Jobs

Grants

CATEGORIES

Content

Design

Development

Others

ABOUT

FAQ

Terms

Privacy Policy

Changelog

Contact Us

SKILL

All Skills ▾

REGION

 Global ▾